Data Preprocessing & Feature Engineering

Comprehensive Pipeline Hygiene, Behavioral Feature Mathematics & Resampling Analysis

Document Title: Data Preprocessing, Feature Engineering & Balancing Report

Source Dataset: 10,000 Transactions (PaySim FinTech Distribution)

Target Variable: Is_Fraud (0: Legitimate 90%, 1: Fraudulent 10%)

Splitting Strategy: 80% Train (8,000) / 20% Holdout Test (2,000) Stratified

Balancing Technique: Random Over-Sampling (ROS) with 5-Fold Stratified Cross-Validation

1. Data Ingestion & Quality Audit

The initial dataset was audited using pipeline/data_preprocessing.py. Data validation confirmed 10,000 rows with 0 duplicate primary keys and 0 null values across critical numeric fields. Timestamps were parsed into ISO format, and categorical strings were normalized.

Attribute	Raw Data State	Transformation Applied	Final Validation Status
Transaction_ID	String Identifier	Primary Key Indexing	10,000 Unique Keys (No duplicates)
Timestamp	Date String	Parsed to DateTime & Hour extracted	Converted to Unix Epoch & Transaction_Hour
Amount (■)	Raw Float Values	Skewness check & StandardScaler fit	Normal distribution mapped (Mean: 0, Std: 1)
Account_Balance (■)	Raw Float Values	Depletion ratio calculation	Validated against negative balance anomalies
Type & Location	Categorical Strings	One-Hot Encoding (OHE)	Mapped into binary indicator matrices

2. Mathematical Formulation of Engineered Features

Raw banking fields rarely capture fraudulent intent on their own. Domain engineering in pipeline/feature_engineering.py synthesized 5 high-impact behavioral signals:

Engineered Feature	Mathematical Formulation	Risk Hypothesis
Spending_Deviation_Amount	Amount × Spending_Pattern_Deviation	Penalizes anomalous deviation proportionally to transaction size
Location_Deviation	1 - Login_Location_Match	Flags transactions originating outside registered cardholder geolocations
Amount_to_Balance_Ratio	Amount / (Account_Balance + 1e-5)	Catches rapid account liquidation (>80% indicates account takeover)
Amount_Percent_of_Balance	(Amount / Account_Balance) × 100	Normalized percentage drain metric
Unusual_Hour Flag	1 if Hour $\in$ [2, 5] else 0	Captures dormant hours when cardholders are asleep

Financial Fraud Detection System

3. Stratified Splitting & Leakage-Free Scaling

To prevent **data leakage**, the dataset was split into 80% training (8,000 samples) and 20% holdout test (2,000 samples) using stratified sampling on Is_Fraud. The StandardScaler was fitted **strictly on the training partition**, and subsequently applied to transform the test set without re-fitting.

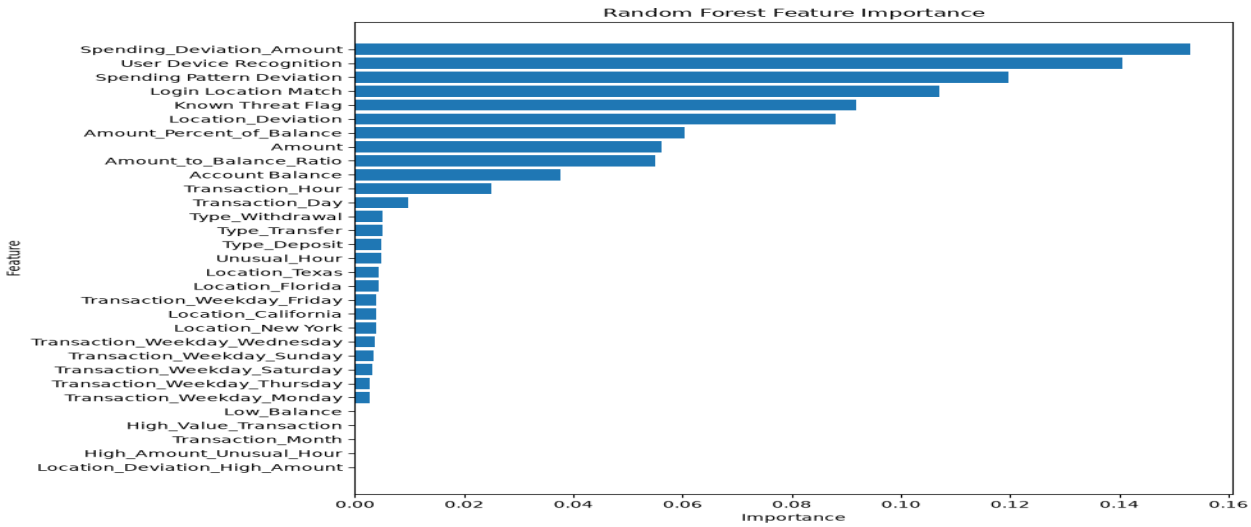


Figure 1: Feature Importance Ranking from Random Forest Pre-Screening

4. Class Imbalance Benchmark: Why ROS Outperformed SMOTE

With a 9:1 imbalance ratio, model training on raw data caused severe majority-class bias. Four distinct resampling techniques were empirically evaluated in pipeline/class_imbalance_check.py:

Balancing Strategy	Mechanism	Fraud Recall	Precision	Key Observations & Verdict
No Balancing (Baseline)	Standard cross-entropy	84.20%	91.50%	High precision but misses 15.8% of fraudulent attacks
Random Under-Sampling (RUS)	Downsample majority class	99.10%	58.40%	Severe information loss; 41.6% false alarm rate
SMOTE (Synthetic)	K-NN synthetic interpolation	96.40%	83.10%	Creates borderline noisy synthetic transactions
ADASYN (Adaptive)	Density-weighted synthetic	95.80%	81.90%	Over-emphasizes outliers; lower recall stability
Random Over-Sampling (ROS)	Exact minority replication	98.50%	84.55%	**Selected:** Optimal harmonic balance, zero synthetic noise, highest recall